JETBRAINS | Kotlin

Functional Programming

kotlinglang.org/education



What is it?

We are already familiar with object-oriented programming (OOP), but Kotlin also borrows concepts from functional programming (FP). FP is a programming paradigm where programs are constructed by **applying** and **composing functions**.

```
var sum = 0
for (item in list) {
   if (item > 0) {
       sum += item * item
   }
}
```

vs

```
list.filter { it > 0 }.map { it * it }.sum()
```

Our approach

FP, like other concepts, has its advantages and disadvantages, but we will focus on its strengths.

Disclaimer: There won't be any deep math or Haskell examples in this lecture. We will look at what we consider to be the most important FP features that can be used in Kotlin

We already know that...

- In Kotlin you can pass functions as the arguments of other functions:

```
fun foo(bar: () → Unit): Unit { ... }
```

- If a function's last argument is a function, then it can be put outside the parentheses:

```
fun baz(start: Int, end: Int, step: (Int) → Unit):
Unit { ... }
baz(23, 42) { println("Magnificent!") }
```

- If a function's only argument is a function, then parentheses can be omitted altogether:

```
foo { println("Kotlin keeps on giving!") }
```

We already know that...

- Lambdas can be assigned to `val`s and reassigned in `var`s:

```
var lambda1: (Int) → Double = { r → r * 6.28 }
val lambda2 = { d: Int → 3.14 * d.toDouble().pow(2) }
lambda1 = lambda2
```

- Lambda expressions can be replaced with function syntax:

```
val sum = fun(a: Int, b: Int): Int = a + b
val sum2 = { a:Int, b: Int → a + b }
```

- Declaring functions inside functions is allowed:

```
fun global() {
      fun local() { ... }
...
      local()

...
}
```

Higher order functions (HOFs)

Functions that take other functions as **arguments** are called higher order functions.

In Kotlin you frequently encounter them when working with collections:

```
list.partition { it % 2 == 0 }
OR  list.partition { x → x % 2 == 0 }
```

Everything Kotlin allows you do with functions, which means that “functions in Kotlin are first-class citizens.”

Higher order functions (HOFs)

In functional programming, functions are designed to be pure. In simple terms, this means they cannot have a state. Loops have an iterator index, which is a state, so say goodbye to *conventional* loops.

```
fun sumIter(term: (Double) → Double, a: Double, next:
(Double) → Double, b: Double): Double {
   fun iter(a: Double, acc: Double): Double = if (a >
b) acc else iter(next(a), acc + term(a))
   return iter(a, 0.0)
}

fun integral(f: (Double) → Double, a: Double, b:
Double, dx: Double): Double {
   fun addDx(x: Double) = x + dx
   return dx * sumIter(f, (a + (dx / 2.0)), ::addDx,
b)
}
```

(This is a LISP program transcribed to Kotlin; nobody actually writes like this)

Higher order functions (HOFs)

Often in the context of FP it is necessary to operate with the following functions: `map`, `filter`, and `fold`.

`map` allows us to perform a function over *each* element in a collection: `val list = listOf(1, 2, 3) list.map { it * it } // [1, 4, 9]`

Higher order functions (HOFs)

Often in the context of FP it is necessary to operate with the following functions: `map`, `filter`, and `fold`.

`map` allows us to perform a function over *each* element in a collection.

```
val list = listOf(1, 2, 3)
list.map { it * it } // [1, 4, 9]
```



Wl

What is the main difference between `map` and `forEach`?

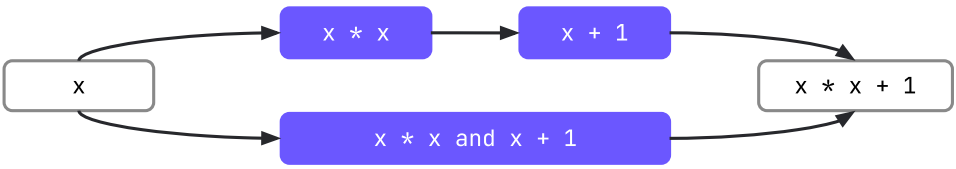
Higher order functions (HOFs)

You can **compose** the functions to perform both operations:

```
val list = listOf(1, 2, 3)

list.map { it * it }.map { it + 1 } // [2, 5, 10]

list.map { it * it + 1 } // [2, 5, 10]
```



NB: to compose complex functions by default you can use sequences, but be careful.

Higher order functions (HOFs)

`filter` returns a list containing only elements that match a given predicate:

```
val list = listOf(1, 2, 3)
list.filter { it % 2 == 0 } // [2]
```

Higher order functions (HOFs)

Our third important function, `fold`, creates a mutable accumulator, which is updated on each round of the `for` and returns one value:

```
val list = listOf(1, 2, 3)
list.fold(0) { acc, x → acc + x } // 6
```

You can implement the `fold` function for any type, for example, you can fold a tree into a string representation.

Higher order functions (HOFs)

There are also right and left `fold`s. They are equivalent if the operation is associative: (a ○ b) ○ c = a ○ (b ○ c), but in any other case they yield different results.

```
val list = listOf(1, 2, 3)
list.fold(0) { acc, x → acc + x }    // (((0 + 1) +
2) + 3) = 6
list.foldRight(0) { x, acc → acc + x } // (1 + (2 +
(3 + 0))) = 6

"PWND".fold("") { acc, x → "${acc}${acc}
$x" }          // PPWPPWNPPWPPWND
"PWND".foldRight("") { x, acc → "${acc}${acc}
$x" }     // DDNDDNWDDNDDNWP
```

Be careful with the order of your lambdas' arguments:

```
list.fold(0) { acc, x → acc -
x } // (((0 - 1) - 2) - 3) = -6
list.foldRight(0) { x, acc → acc -
x } // (-1 + (-2 + (0 - 3))) = -6
list.foldRight(0) { acc, x → acc - x } // (1 - (2 -
(3 - 0))) = 2
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

val result = string
    .split(" ", "-", ".", System.lineSeparator())
    .filter { it.isNotEmpty() }
    .map { it.lowercase() }
    .groupingBy { it }
    .eachCount()
    .toList()
    .sortedBy { (_, count) → count }
    .reversed()
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

  val result = string
      .split(" ", "-", ".", System.lineSeparator())
```

```
[One, one, was, a, race, horse, , Two, two, was, one,
too, , One, one, won,
one, race, , Two, two, won, one, too, , ]
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

  val result = string
      .split(" ", "-", ".", System.lineSeparator())
      .filter { it.isNotEmpty() }
```

```
[One, one, was, a, race, horse, Two, two, was, one,
too, One, one, won,
one, race, Two, two, won, one, too]
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

  val result = string
      .split(" ", "-", ".", System.lineSeparator())
      .filter { it.isNotEmpty() }
      .map { it.lowercase() }
```

```
[one, one, was, a, race, horse, two, two, was, one,
too, one, one,
won, one, race, two, two, won, one, too]
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

  val result = string
      .split(" ", "-", ".", System.lineSeparator())
      .filter { it.isNotEmpty() }
      .map { it.lowercase() }
      .groupingBy { it }
      .eachCount()
```

OR

```
        string
            .split(...)
            .filter { it.isNotEmpty() }
            .groupBy({ it.lowercase() }, { it })
            .mapValues { (key, value) →
                value.size
            }
```

```
{one=7, was=2, a=1, race=2, horse=1, two=4, too=2,
won=2}
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

  val result = string
      .split(" ", "-", ".", System.lineSeparator())
      .filter { it.isNotEmpty() }
      .map { it.lowercase() }
      .groupingBy { it }
      .eachCount()
      .toList()
```

```
[(one, 7), (was, 2), (a, 1), (race, 2), (horse, 1),
(two, 4),
(too, 2), (won, 2)]
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()

  val result = string
      .split(" ", "-", ".", System.lineSeparator())
      .filter { it.isNotEmpty() }
      .map { it.lowercase() }
      .groupingBy { it }
      .eachCount()
      .toList()
      .sortedBy { (_, count) -> count }
```

```
[(a, 1), (horse, 1), (was, 2), (race, 2), (too, 2),
(won, 2), (two, 4), (one, 7)]
```

Higher order functions (HOFs)

```
val string = """
    One-one was a race horse.
    Two-two was one too.
    One-one won one race.
    Two-two won one too.
""".trimIndent()
```

```
            val result = string
                .split(" ", "-", ".", System.lineSeparator())
                .filter { it.isNotEmpty() }
                .map { it.lowercase() }
                .groupingBy { it }
                .eachCount()
                .toList()
                .sortedBy { (_, count) → count }
                .reversed()
```

```
[(one, 7), (two, 4), (won, 2), (too, 2), (race,
2), (was, 2), (horse, 1), (a, 1)]
```

OR

OR

```
                    string
                        .allFunnyFuncs(...)
                        .toList()
                        .sortedWith { l, r →
                            r.second - l.second
                        }
```

```
                string
                    .allFunnyFuncs(...)
                    .toList()
                    .sortedByDescending { (_, c) →
                        c
                    }
```

Higher order functions (HOFs)

Lambdas are not the only functions that can be passed as arguments to functions expecting other functions, as *references* to already defined functions can be as well:

```
fun isEven(x: Int) = x % 2 == 0

val isEvenLambda = { x: Int → x % 2 ==
0 }
```

Same results, different calls:

- `list.partition { it % 2 == 0 }`
- `list.partition(::isEven)` `// function reference`
- `list.partition(isEvenLambda)` `// pass lambda by name`

Lazy computations

Consider the following code:

```
fun <F> withFunction(
    number: Int, even: F, odd: F
): F = when (number % 2) {
    0 → even
    else → odd
}

withFunction(4, println("even"), println("odd"))
```

What will be printed to the console?

Lazy computations

Consider the following code:

```
fun <F> withFunction(
    number: Int, even: F, odd: F
): F = when (number % 2) {
    0 → even
    else → odd
}

withFunction(4, println("even"),
println("odd"))
```

What will be printed into console?`even odd`

Arguments of the `withFunction` function will be evaluated **before** its body is executed (eager execution).

Lazy Deferred computations

Consider the following code:

```
fun <F> withLambda(
    number: Int, even: () → F, odd: () → F
): F = when (number % 2) {
    0 → even()
    else → odd()
}

withLambda(4, { println("even") }, { println("odd") })
```

It will print just `even` into the console because of the ~~lazy~~ deferred computations.

Operator overloading

Kotlin has extension functions that you can use to override operators, for example the `iterator`. That is, you do not need to create a new entity that inherits from the Iterable interface, as you would in OOP code.

```
class MyIterable<T> : Iterable<T>
{ // you need access to the sources of MyIterable
    override fun iterator(): Iterator<T> {
        TODO("Not yet implemented")
    }
}
```

VS

```
class A<T>
operator fun <T> A<T>.iterator(): Iterator<T> =
TODO("Not yet implemented")
```

One last thing...

Is this code correct?

```
enum class Color {
    WHITE,
    AZURE,
    HONEYDEW
}

fun Color.getRGB() = when (this) {
   Color.WHITE → "#FFFFFF"
   Color.AZURE → "#F0FFFF"
   Color.HONEYDEW → "F0FFF0"
}
```

One last thing...

Is this code correct? **Yes**, because the compiler knows **all** of the possible values.

```
enum class Color {
    WHITE,
    AZURE,
    HONEYDEW
}

fun Color.getRGB() = when (this) {
   Color.WHITE → "#FFFFFF"
   Color.AZURE → "#F0FFFF"
   Color.HONEYDEW → "F0FFF0"
}
```

One last thing...

What is about this example?

```
sealed class Color

class WhiteColor: Color()
class AzureColor: Color()
class HoneydewColor: Color()

fun Color.getRGB() = when (this) {
   is WhiteColor → "#FFFFFF"
   is AzureColor → "#F0FFFF"
   is HoneydewColor → "F0FFF0"
}
```

One last thing...

What about this example? Once again, the answer is **yes**, because the compiler knows about **all** possible children of the `Color` class at the compilation stage and no new classes can appear.

```
sealed class Color

class WhiteColor: Color()
class AzureColor: Color()
class HoneydewColor: Color()

fun Color.getRGB() = when (this) {
   is WhiteColor → "#FFFFFF"
   is AzureColor → "#F0FFFF"
   is HoneydewColor → "F0FFF0"
}
```

One last thing...

Consider the following code:

```
sealed class Color
class WhiteColor(val name: String): Color()
class AzureColor(val name: String): Color()
class HoneydewColor(val name: String): Color()
```

We have the common part in **all** classes and we know that these are the **only** possible subclasses. Let's move this code into the base class.

One last thing...

```
sealed class Color
class WhiteColor(val name: String): Color()
class AzureColor(val name: String): Color()
class HoneydewColor(val name: String): Color()
```



```
sealed class NewColor(val name: String)
class WhiteColor(name: String): NewColor(name)
class AzureColor(name: String): NewColor(name)
class HoneydewColor(name: String): NewColor(name)
```

Actually, we have **equivalent** classes, i.e. *each* function for the first version can be rewritten as the *second* one.

One last thing...

```
sealed class Color
class WhiteColor(val name: String): Color()
class AzureColor(val name: String): Color()
class HoneydewColor(val name: String): Color()
```



```
sealed class NewColor(val name: String)
class WhiteColor(name: String): NewColor(name)
class AzureColor(name: String): NewColor(name)
class HoneydewColor(name: String): NewColor(name)
```

```
fun Color.getUserRGB() = when (this) {
    is WhiteColor → "${this.name}: #FFFFFF"
    is AzureColor → "${this.name}: #F0FFFF"
    is HoneydewColor → "${this.name}: F0FFF0"
}
```

```
fun NewColor.getUserRGB() = when (this) {
    is WhiteColor → "${this.name}: #FFFFFF"
    is AzureColor → "${this.name}: #F0FFFF"
    is HoneydewColor → "${this.name}: F0FFF0"
}
```

In the first function, we have smart casts, but in the second one we don't have them.

One last thing...

```
sealed class Color
class WhiteColor(val name: String): Color()
class AzureColor(val name: String): Color()
class HoneydewColor(val name: String): Color()
```



```
sealed class NewColor(val name: String)
class WhiteColor(name: String): NewColor(name)
class AzureColor(name: String): NewColor(name)
class HoneydewColor(name: String): NewColor(name)
```

```
fun Color.getUserRGB() = when (this) {
    is WhiteColor → "${this.name}: #FFFFFF"
    is AzureColor → "${this.name}: #F0FFFF"
    is HoneydewColor → "${this.name}: F0FFF0"
}
```

```
fun NewColor.getUserRGB() = when (this) {
    is WhiteColor → "${this.name}: #FFFFFF"
    is AzureColor → "${this.name}: #F0FFFF"
    is HoneydewColor → "${this.name}: F0FFF0"
}
```

Math time! We can actually rewrite this in math terms:

`WhiteColor` * `String` + ... + `HoneydewColor` * `String` ≈ `String` * (`WhiteColor` + ... + `HoneydewColor`)

One last thing...

Math time! We can actually rewrite this in math terms: `WhiteColor * String + ... + HoneydewColor * String ≃ String * (WhiteColor + ... + HoneydewColor)`

This is possible because we are actually operating with **algebraic data types*** and can use their properties.

**This is not entirely true, but for most cases with sealed classes it works.*

Final thought

FP in Kotlin does not kill OOP. Each of the concepts brings its own advantages and disadvantages, and it is important to combine them in order to get concise, readable and understandable code!

If you are interested in the topic of FP in Kotlin for a more detailed study, come here: https://arrow-kt.io/

JETBRAINS | Kotlin

Thank you

